Uniform Sampling from the Unit Simplex

Bernhard von Stengel

February 3, 2025

Abstract

We describe two simple methods to choose a point uniformly from the unit simplex, and how round that point to a vector of fractions with a fixed limited denominator.

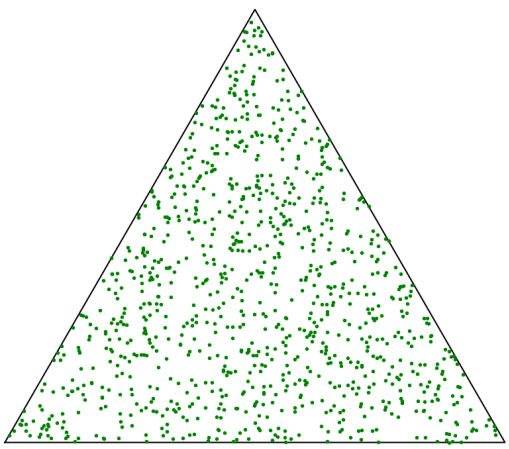
1 Uniform sampling from the unit simplex

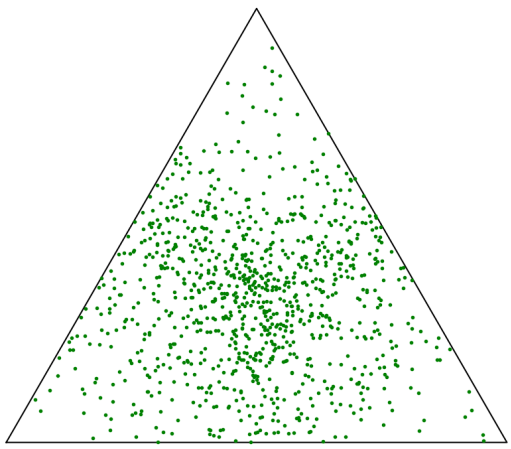
The *unit simplex* Δ_n is the convex hull of the n unit vectors in $\mathbb{R}^n$, that is,

$$\Delta_n = \{\, (x_1, \ldots, x_n) \mid x_1 + \cdots + x_n = 1,\ x_i \geq 0\ (1 \leq i \leq n) \,\}\,.$$

As a geometric object, Δ_n is a subset of $\mathbb{R}^n$, but has *dimension* $n-1$ due to the equation. For example, Δ_3 is an equilateral triangle (dimension 2), and Δ_4 is a regular tetrahedron (dimension 3).

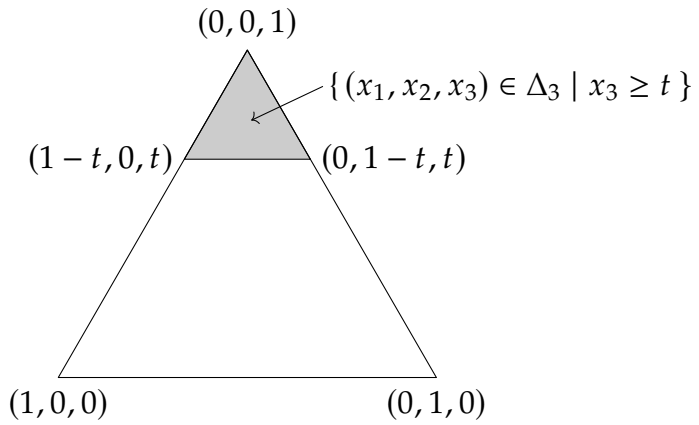
We want to choose a point $x = (x_1, \ldots, x_n)$ in Δ_n uniformly at random. The naive method of choosing n numbers uniformly from the unit interval and dividing them by their sum concentrates the points closer to the center of the simplex, as in the following picture on the right for $n = 3$:



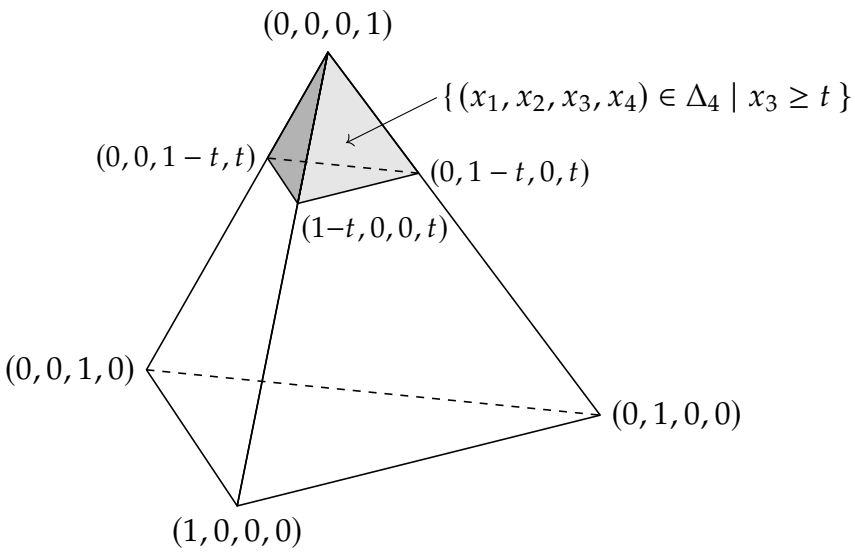


The uniform distribution on the unit simplex is also known as the "flat Dirichlet distribution".[1] Instead of using a software package that generates points from this distribution, we generate the coordinates $x_1, \ldots, x_n$ of a point x in Δ_n recursively, starting with x_n.

If x_n has value t in $[0, 1]$, this "cuts off" a small simplex of height $1 - t$ from Δ_n, as shown here for $n = 3$:



and $n = 4$:



The volume of that cut-off simplex is the same as that of Δ_n multiplied by $(1-t)^{n-1}$, because it is the same $(n-1)$-dimensional geometric object as Δ_n with each dimension reduced from 1 to $1-t$. The probability of x being in that small simplex is consequently given by that volume, that is, for our uniform sampling

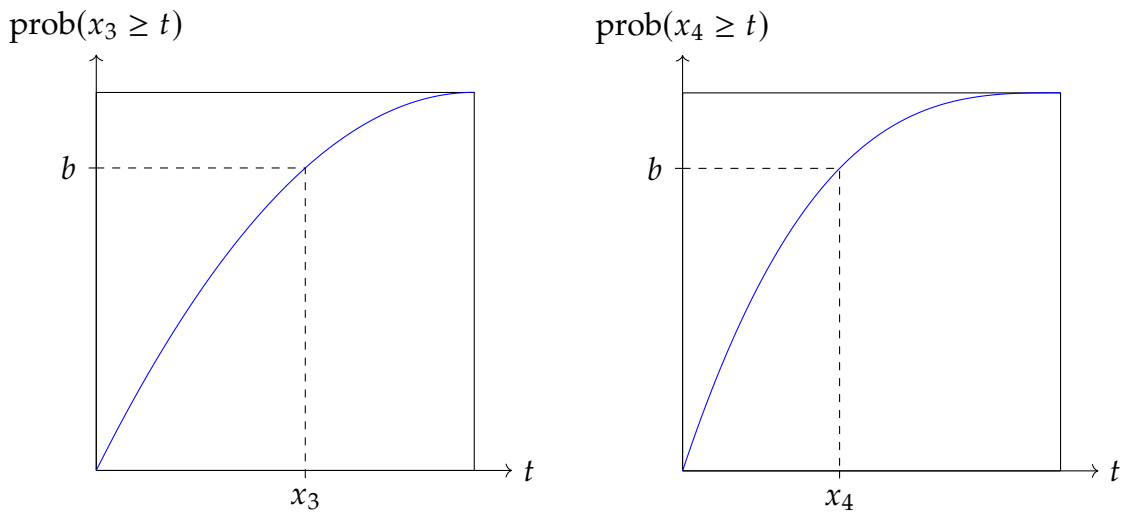
$$\text{prob}(x_n \geq t) = (1-t)^{n-1} \,. \tag{1}$$

[1]See section XI.4 of Devroye, L. (1986), Non-Uniform Random Variate Generation. Springer, New York. https://luc.devroye.org/rnbookindex.html

The cumulative distribution function $\text{prob}(x_n \le t)$ is therefore given by

$$\text{prob}(x_n \le t) = 1 - (1-t)^{n-1}, \tag{2}$$

shown here for $n = 3$ and $n = 4$:



These pictures describe how to find the proper random value of x_n from the cumulative distribution function (**??**): Choose the vertical value b of the function $1-(1-t)^{n-1}$ uniformly at random from $[0,1]$ and then determine $x_n = t$ according to $b = 1-(1-t)^{n-1}$, that is,

$$(1-b)^{1/(n-1)} = 1 - x_n \tag{3}$$

where we can just as well choose $1-b$ rather than b uniformly from $[0,1]$, giving

$$1 - x_n = b^{1/(n-1)}. \tag{4}$$

With x_n randomly chosen in that way, the remaining coordinates $x_1, \ldots, x_{n-1}$ belong to a scaled version of Δ_{n-1} (the "bottom" of the "cut-off tip" of Δ_n) where the side of each edge is shrunk from length 1 to $1 - x_n$. We then determine x_{n-1} by sampling from the simplex Δ_{n-1} of the next-smaller dimension, except that all coordinates are multiplied by $1 - x_n$, shown as `f` in the Python method below. The method computes $(x_1, \ldots, x_n)$, per standard convention, as a list with entries `x[0]`, ..., `x[n-1]`. It first computes `x[n-1]`, and then recursively `x[n-2]`, ..., `x[0]` in successively smaller dimensions, except that these components have a smaller sum given by the variable `factor`, which at the beginning of each iteration of the `while` loop equals $x_0 + \cdots + x_i$, starting with $i = n-1$. The method iterates down to $i = 1$, and finally sets `x[0]` as the remaining value of `factor`.

```
import random
def randInSimplex(n):
    x = [0.0]*n
    factor = 1.0
    i = n-1
    while i>0:
        b = random.uniform(0, 1)
        f = b**(1/i)
        x[i] = factor * (1-f)
        factor *= f
        i -= 1
    x[0] = factor
    return x
```

2 Logarithms of i.i.d. uniform variables

Surprisingly, we can also generate a uniform distribution on the unit simplex Δ_n by the following fully symmetric method: Choose independently n variables $y_1, \ldots, y_n$ uniformly at random from the interval $[0, 1]$ (assuming that 0 and 1 will not be chosen), take their *logarithms*, and normalize them so that they sum to 1 (the base of the logarithm doesn't matter). That is, $x = (x_1, \ldots, x_n)$ is computed from $(y_1, \ldots, y_n)$ via

$$x_i = \frac{\log y_i}{\sum_{k=1}^{n} \log y_k} \qquad (1 \leq i \leq n). \tag{5}$$

The claim is that then x is uniformly chosen from Δ_n. The corresponding Python method is this:

```
import math
import random
def randInSimplex(n):
    y = [0.0]*n
    sum = 0.0
    for i in range(n):
        y[i] = math.log(random.uniform(0, 1))
        sum += y[i]
    return [z/sum for z in y]
```

Apparent from similar pictures to those above, this clearly works. But why? We prove this with standard first-year mathematics.[2]

The vector $y = (y_1, \ldots, y_n)$ is chosen uniformly from the n-dimensional unit cube $[0,1]^n$. With x as defined in (**??**), we want to show (**??**).

By (**??**), we have the following equivalences (note that all logarithms are negative):

$$\begin{aligned}
x_n &\geq t \\
\log y_n / \textstyle\sum_{k=1}^n \log y_k &\geq t \\
\log y_n &\leq t \textstyle\sum_{k=1}^n \log y_k \\
\log y_n (1-t) &\leq t \textstyle\sum_{k=1}^{n-1} \log y_k \\
\log y_n &\leq \textstyle\sum_{k=1}^{n-1} \frac{t}{1-t} \log y_k \\
y_n &\leq \textstyle\prod_{k=1}^{n-1} y_k^{t/(1-t)}
\end{aligned}$$

where the equation

$$y_n = \prod_{k=1}^{n-1} y_k^{t/(1-t)}$$

defines y_n as a function of $y_1, \ldots, y_{n-1}$, that is, a surface in the cube $[0,1]^n$ that is "above" the facet of the cube given by the points $(y_1, \ldots, y_{n-1}, 0)$. The cube itself has volume 1, hence $\text{prob}(x_n \geq t)$ is the volume below that surface, given by the following multiple integral, where we use that $t/(1-t) = 1/(1-t) - 1$ (it helps to visualize first the case $n = 2$):

$$\begin{aligned}
\text{prob}(x_n \geq t) &= \textstyle\int_0^1 \int_0^1 \cdots \int_0^1 \prod_{k=1}^{n-1} y_k^{t/(1-t)} dy_{n-1} \cdots dy_2\, dy_1 \\
&= \left(\textstyle\int_0^1 z^{t/(1-t)} dz \right)^{n-1} \\
&= \left(\left[\frac{1}{1/(1-t)} z^{1/(1-t)} \right]_{z=0}^{z=1} \right)^{n-1} \\
&= (1-t)^{n-1}
\end{aligned}$$

which proves (**??**). Because (**??**) is symmetric in all components, we have more generally for all $i = 1, \ldots, n$ that

$$\text{prob}(x_i \geq t) = (1-t)^{n-1}\,.$$

[2]This was asked at https://math.stackexchange.com/questions/4235192/prove-this-method-generates-uniform-points-on-the-simplex, with my answer now added there. See also (without proof) http://blog.geomblog.org/2005/10/sampling-from-simplex.html

3 Rounding points in the simplex to small fractions

The points $(x_1, \ldots, x_n)$ in Δ_n are floating-point numbers. We use them as mixed strategies that define starting points in computations for finding Nash equilibria in two-player games, where we need fractions whose denominator does not exceed a certain size h, say. Hence, we need to round each x_i to a fraction p_i/h with numerator p_i such that $p_1 + \cdots + p_n = h$. This problem has been studied as the *apportionment* of an integer number p_i of seats to each party with vote fraction x_i in a house of parliament with h seats in total.[3] There are various methods for this, which can vary significantly if h is small, such as $h = 10$ (see below). For our application, typically $h = 10000$, and the choice of method is irrelevant.

We choose the following: Let $q_i = x_i\, h$ be the ideal quota for each party $i = 1, \ldots, n$, and consider its integer part $\lfloor q_i \rfloor$ as the minimum number of seats for that party. The resulting total remainder $r = \sum_{i=1}^{n}(q_i - \lfloor q_i \rfloor) = h - \sum_{i=1}^{n} \lfloor q_i \rfloor$ fulfills $0 \le r < n$. We set $p_i = \lfloor q_i \rfloor + 1$ for the r parties with the largest remainders $q_i - \lfloor q_i \rfloor$, and $p_i = \lfloor q_i \rfloor$ for rest. Then $p_1 + \cdots + p_n = h$ as required.

For example, let $n = 3$ and $(x_1, x_2, x_3) = (0.756, 0.164, 0.08)$ and $h = 10$. Then $(q_1, q_2, q_3) = (7.56, 1.64, 0.8)$ and $r = 2$. The described method of largest remainders gives $(p_1, p_2, p_3) = (7, 2, 1)$. The method is known to favor smaller parties. However, it also has a non-monotonicity called the "Alabama paradox" where increasing h can lead to a lower value of some p_i (which is irrelevant for our application where h is fixed). The different "d'Hondt" method favors large parties. For the above example, it gives $(p_1, p_2, p_3) = (9, 1, 0)$ where both extra seats go to the largest party, in excess of its "natural" quota that only allows $p_1 = 7$ or $p_1 = 8$.

[3] Balinski, M. L. and H. P. Young (1994), Apportionment. In: S. M. Pollock et al, eds, *Handbooks in Operations Research and Management Science* Vol. 6, Chapter 15, 529–560.